ASSIGNMENT 5
Apply the KNN Algorithm for Classification

1. Aim

To implement the K-Nearest Neighbours (KNN) algorithm for classifying Iris flower species.

2. Objectives

- To understand the concept of KNN classification.
- To evaluate the classifier using accuracy and a confusion matrix.
- To apply feature standardisation.
- To implement KNN using Python.
- To learn the role of the number of neighbours.

3. Theory

K-Nearest Neighbours (KNN) is a supervised machine-learning algorithm used for classification and regression. In classification, it predicts the category of a new observation by examining the categories of nearby training observations.

KNN is an instance-based learning algorithm because it stores the training examples and uses them when making predictions.

The working process is as follows:

1. Select the value of K.
2. Calculate the distances between a new observation and the training observations.
3. Assign the most common class to the new observation.

4. Count the class labels among those neighbours.
5. Select the K nearest neighbours.
6. Calculate the distances between a new observation and the training observations.

For example, if K is 5, the algorithm considers the five nearest training examples and predicts the class receiving the majority vote.

Feature scaling is important because KNN relies on distances. If one feature has a much larger range than another, it may dominate the distance calculation.

A confusion matrix compares actual labels with predicted labels. Correct predictions appear along the main diagonal, while other entries show misclassifications.

In this assignment, KNN is applied to the Iris dataset using five neighbours.

4. Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Divide the dataset into training and testing sets.
4. Standardise the numerical features.
5. Create a KNN classifier with five neighbours.
6. Train the classifier.
7. Predict the test labels.
8. Calculate accuracy and display the confusion matrix.

5. Explanation of Important Code

- KNeighborsClassifier(n_neighbors=5) creates a KNN classifier that considers five neighbours.

- StandardScaler() standardises the input features.
- make_pipeline() ensures that scaling and classification are performed in the correct order.
- fit() prepares the model using training data.
- predict() predicts the species for test observations.
- accuracy_score() calculates classification accuracy.
- confusion_matrix() shows correct and incorrect classifications.

6. Applications

1. Classifying flowers and biological samples.
2. Pattern recognition.
3. Similarity-based recommendation systems.
4. Handwriting classification after feature extraction.
5. Classification of numerical sensor measurements.

7. Advantages

1. Easy to understand and implement.
2. Does not require a complex training process.
3. Can represent nonlinear classification boundaries.
4. Can be used for classification and regression.

8. Limitations

1. Predictions can be slow for large training datasets.
2. Performance depends on the selected value of K.

3. Sensitive to feature scaling and irrelevant features.
4. Noise and outliers may affect neighbour selection.

9. Conclusion

The K-Nearest Neighbours algorithm was implemented using five neighbours to classify Iris flowers. Feature standardisation was applied before classification, and the model was evaluated using accuracy and a confusion matrix.